GOLDEN GATE
UNIVERSITY

Lecture 6: Linux Lab

# From Logic Gates to the Operating System

## Computer Organization and Digital Foundations: hands-on with Linux commands


**Dr. Yatish Bathla**

**PhD.(Information Technology), Obuda University, Budapest**

# Today's Lab Roadmap

1. Build gates, adders, a shift register and a counter with bash arithmetic
2. Run a tiny CPU in bash and trace fetch, decode and execute
3. Inspect the CPU and the kernel of your Linux container
4. Observe processes, files and system calls on a live OS

## 60-minute plan

| | |
|---|---|
| **0-5 min** | Setup and helper file (slide 3) |
| **5-25 min** | Digital logic in bash (4-7) |
| **25-40 min** | CPU: inspect and simulate (8-10) |
| **40-55 min** | OS: kernel, processes, syscalls (11-13) |
| **55-60 min** | Practice and summary (14-15) |

# Setup: Linux Container and Helper File

**Terminal**

```
# 1. Open a shell in your container
$ docker ps
$ docker exec -it <container_name> bash

# 2. Check that you are in bash
$ echo $BASH_VERSION
5.1.16(1)-release

# 3. Make a lab folder
$ mkdir -p ~/lab && cd ~/lab
$ pwd
/root/lab
```

**Helper: print a number in binary**

```
$ cat > lib.sh << 'EOF'
bin() {         # bin NUMBER BITS
  local i
  for ((i=$2-1; i>=0; i--)); do
    printf %d $(( ($1>>i)&1 ))
  done; echo
}
EOF
$ source lib.sh
$ bin 5 4
0101
```

**If a command is missing later**

apt-get update && apt-get install -y procps strace

All work today uses only Linux commands: no simulator to install.

# Logic Gates with Shell Operators

**Type this in the terminal**

```
echo "A B AND OR XOR NAND NOR"
for A in 0 1; do for B in 0 1; do
  and=$((A&B)); or=$((A|B)); xor=$((A^B))
  echo "$A $B  $and   $or  $xor   $((1-and))    $((1-or))"
done; done
```

| A | B | AND | OR | XOR | NAND | NOR |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

### Shell operators

A & B = AND  A | B = OR  A ^ B = XOR
1-x = NOT for one bit (~ flips every bit)
x << 1 = shift left  x >> 1 = shift right

### Expected output

Your loop prints exactly this table.
XOR = 1 when inputs differ.
AND = 1 only when both inputs are 1.

A gate is a Boolean operator: a loop tests every input row.

# Half Adder and Full Adder

## Half adder

```
                    echo "x y  C S"
for x in 0 1; do for y in 0 1; do
  echo "$x $y  $((x&y)) $((x^y))"
done; done
```

**Output**

```
x y  C S
0 0  0 0
0 1  0 1
1 0  0 1
1 1  1 0
```

**1 + 1 = 10 in binary**

sum 0, carry 1

## Full adder (adds a carry-in z)

```
                    echo "x y z  C S"
for x in 0 1; do for y in 0 1; do for z in 0 1; do
  S=$((x^y^z)); C=$(( (x&y) | (z&(x^y)) ))
  echo "$x $y $z  $C $S"
done; done; done
```

**Output**

```
x y z  C S
0 0 0  0 0
0 0 1  0 1
0 1 0  0 1
0 1 1  1 0
1 0 0  0 1
1 0 1  1 0
1 1 0  1 0
1 1 1  1 1
```

# 4-Bit Ripple-Carry Adder

**Type this in the terminal (it stays defined until you exit)**

```
add4() {
  local a=$1 b=$2 c=0 s=0 i x y
  for i in 0 1 2 3; do
    x=$(( (a>>i)&1 )); y=$(( (b>>i)&1 ))
    s=$(( s | ((x^y^c) << i) ))
    c=$(( (x&y) | (c&(x^y)) ))
  done
  echo "sum=$s carry_out=$c"
}
add4 7 5
add4 11 6
add4 15 1
```

**Output**

```
sum=12 carry_out=0    # 7 + 5
sum=1 carry_out=1     # 11 + 6 = 17
sum=0 carry_out=1     # 15 + 1 = 16
```

## How it works

Each pass of the loop is one full adder for bit i. The new carry c goes into the next pass.

## Try

add4 9 8  and  add4 3 3

# Memory in the Shell: Flip-Flop, Shift Register, Counter

### D flip-flop

```
                Q=0
for D in 1 0 1; do
  echo "D=$D  before: Q=$Q"
  Q=$D          # clock edge
  echo "       after:  Q=$Q"
done
```

**Output**

```
D=1  before: Q=0
       after:  Q=1
D=0  before: Q=1
       after:  Q=0
D=1  before: Q=0
       after:  Q=1
```

### 4-bit shift register

```
                R=0
for IN in 1 1 0 1; do
  R=$(( ((R<<1)|IN) & 15 ))
  echo -n "in=$IN  R="
  bin $R 4
done
```

**Output**

```
in=1  R=0001
in=1  R=0011
in=0  R=0110
in=1  R=1101
```

### 3-bit counter

```
                C=0
for t in $(seq 1 10); do
  C=$(( (C+1) % 8 ))
  echo -n "tick $t: "
  bin $C 3
  sleep 0.2
done
```

**Output**

```
tick 1: 001
tick 2: 010
tick 3: 011
...
tick 7: 111
tick 8: 000
tick 9: 001
```

Memory = a value that changes only at the clock tick; registers update every tick.

# Your CPU and the Performance Equation

**Inspect the CPU (your values will differ)**

```
$ nproc
8
$ lscpu | grep -E 'Arch|Model name|Byte'
Architecture:  x86_64
Byte Order:    Little Endian
Model name:    <your CPU model>
$ grep -m1 MHz /proc/cpuinfo
cpu MHz : 2994.3
```

**Reading it**

Architecture = ISA, CPU(s) = cores, MHz = clock (T = 1/f).

**CPU time = IC x CPI x T**

```
cpu_time() {
  awk -v ic=$1 -v cpi=$2 -v f=$3 \
    'BEGIN{print ic*cpi/f " s"}'
}
cpu_time 2e9 1.5 2e9
cpu_time 2e9 1.0 2e9
```

**Output**

```
1.5 s    # IC = 2e9, CPI = 1.5, 2 GHz
1 s      # CPI lowered to 1.0 (pipelining)
```

**Your turn**

IC = 1e9, CPI = 2, clock = 2 GHz: run cpu_time.

# A Tiny CPU in bash: The Program

Type this exactly

```
$ cat > cpu.sh << 'EOF'
M[100]="LOAD 200"; M[101]="ADD 201"
M[102]="STORE 202"
M[200]=5; M[201]=7; M[202]=0
PC=100; ACC=0
while [ $PC -le 102 ]; do
  IR=${M[$PC]}; PC=$((PC+1))    # FETCH
  set -- $IR; OP=$1; ADDR=$2    # DECODE
  case $OP in                   # EXECUTE
    LOAD)  ACC=${M[$ADDR]} ;;
    ADD)   ACC=$((ACC+M[$ADDR])) ;;
    STORE) M[$ADDR]=$ACC ;;
  esac
  echo "IR=$IR PC=$PC ACC=$ACC M[202]=${M[202]}"
done
EOF
```

1 **Fetch**
IR = M[PC], then PC = PC + 1

2 **Decode**
Split IR into opcode and address

3 **Execute**
case picks the operation: ALU or memory

Registers: PC, IR, ACC. Memory: M[ ]

# A Tiny CPU in bash: Run and Trace

```
$ bash cpu.sh
IR=LOAD 200 PC=101 ACC=5 M[202]=0
IR=ADD 201 PC=102 ACC=12 M[202]=0
IR=STORE 202 PC=103 ACC=12 M[202]=12
```

| After | PC | ACC | M[202] |
|---|---|---|---|
| **start** | 100 | 0 | 0 |
| **LOAD 200** | 101 | 5 | 0 |
| **ADD 201** | 102 | 12 | 0 |
| **STORE 202** | 103 | 12 | 12 |

### Your turn: task 1

Change the data and predict the answer first.

```
$ sed -i 's/M\[201\]=7/M[201]=10/' cpu.sh
$ bash cpu.sh | tail -1
```

### Task 2

Add a new line to the case block:
SUB) ACC=$((ACC-M[$ADDR])) ;;
Then change ADD 201 to SUB 201 and run.

Every instruction is fetch, decode, execute: your script did what a CPU does.

# The Kernel and Processes

**Kernel and distribution**

```
$ uname -sr
Linux 6.x.x    # the host's kernel
$ head -2 /etc/os-release
PRETTY_NAME="Ubuntu 22.04 LTS"
NAME="Ubuntu"
# Linux = kernel, Ubuntu = distribution
```

### Container or virtual machine?

A container shares the host kernel, so uname -r matches the host. A virtual machine runs its own kernel.

**Processes (your PIDs will differ)**

```
$ echo $$          # PID of this shell
$ sleep 300 &      # start a child
$ ps -o pid,ppid,cmd
  PID  PPID CMD
    1     0 bash
   30     1 sleep 300
   31     1 ps -o pid,ppid,cmd
$ kill %1          # end the child
```

### fork + exec

Every command you type becomes a child of the shell.

# Everything Is a File: /proc and /dev

**Try each command (numbers will differ)**

```
$ cat /proc/uptime
5231.77 41020.13
$ head -3 /proc/meminfo
MemTotal:       16240000 kB
MemFree:         8120000 kB
MemAvailable:   12500000 kB
$ cat /proc/loadavg
0.15 0.10 0.05 1/180 31
$ ls /dev | head -5
$ head -c 4 /dev/urandom | od -An -tx1
 a3 1f 9c 07
$ head -5 /proc/modules     # may be empty here
```

### /proc

Information made live by the kernel. Nothing is stored on disk.

### /dev

Devices look like files: read and write them as usual.

### Kernel modules

Linux is monolithic, but drivers can be loaded as modules.

Everything is a file: one simple interface between programs and the kernel.

# User Mode, Kernel Mode and System Calls

**Try each command (numbers will differ)**

```
$ strace -e trace=write echo hello > /dev/null
write(1, "hello\n", 6)    = 6
$ strace -c ls > /dev/null
% time  seconds  calls  syscall
  ...   ...      ...    openat, read, write, close
$ time ls -R /usr > /dev/null
real   0m0.210s
user   0m0.060s
sys    0m0.140s
$ mount -t tmpfs none /mnt
mount: /mnt: permission denied
```

**USER MODE**

your program, library (printf)

**system call: trap into the kernel**

**KERNEL MODE**

file system, drivers, memory manager

### Reading the results

user = time in your code, sys = time in the kernel. Only the kernel may do privileged things, so mount is refused.

# Practice Set: Work It Out

| # | Task | Hint | Your answer |
|---|---|---|---|
| **1** | Which gate gives 1 only when its inputs differ? | Truth table slide | |
| **2** | Run add4 9 8. What are sum and carry_out? | 9 + 8 = 17 | |
| **3** | Shift register inputs 1, 0, 1, 1: final value of R? | Change the loop list | |
| **4** | 3-bit counter: what is the value after 12 ticks? | 12 mod 8 | |
| **5** | cpu.sh with M[201]=10: final ACC? | sed command | |
| **6** | In the time output, which number is kernel-mode time? | user, sys, real | |

**Challenge:** run cpu.sh with the new SUB instruction so that M[202] = M[200] - M[201]. What is the result?

# Common Problems and Summary

| | |
|---|---|
| ✗ `A = 1` (spaces around =) | ✓ `A=1` no spaces in bash assignments |
| ✗ `ps: command not found` | ✓ `apt-get update && apt-get install -y procps strace` |
| ✗ `bin: command not found` | ✓ `source lib.sh` in every new shell |
| ✗ `sh cpu.sh` (arrays fail) | ✓ `bash cpu.sh` run scripts with bash |

**Lab summary**

Gates and adders are operators. Flip-flops and counters are values updated at a tick. A CPU is a fetch-decode-execute loop. The OS gives processes, files and system calls through the kernel.

Save your work: history > lab_notes.txt then exit